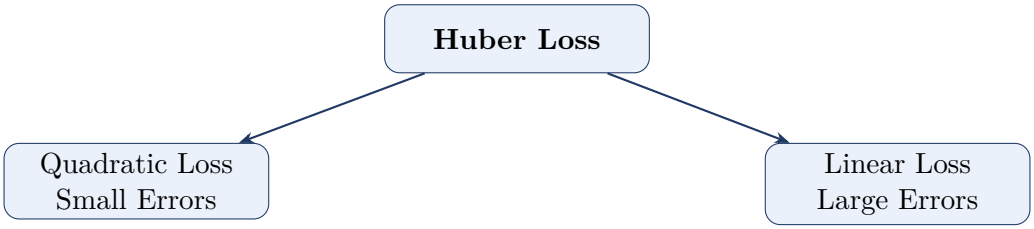
B.TECH COMPUTER SCIENCE AND ENGINEERING

MACHINE LEARNING

HUBER LOSS

A Robust Loss Function for Regression



Machine Learning Report

Submitted By: Kalash Kandari

SAP ID: 590026638

Programme: B.Tech CSE

Subject: Machine Learning

Topic: Huber Loss

Date: 15 October 2026

1 Introduction

Loss functions are fundamental components of machine learning. They measure the difference between the actual value and the value predicted by a model. During training, an optimization algorithm adjusts the parameters of the model to minimize the selected loss function.

For regression problems, two common choices are Mean Squared Error (MSE) and Mean Absolute Error (MAE). Squared error penalizes large errors strongly because the residual is squared. This property is useful when large errors should receive a strong penalty, but it also makes the model sensitive to outliers. Absolute error increases linearly and is therefore less affected by extreme observations.

Huber Loss provides a compromise between these approaches. It behaves quadratically when the prediction error is small and linearly when the error is large. Therefore, it preserves the useful smooth behaviour of squared error around zero while reducing the influence of extreme residuals.

Huber Loss was introduced in the context of robust statistics by Peter J. Huber. Today, Huber-type loss functions are used in robust regression, machine learning, deep learning, computer vision, forecasting, and applications containing noisy observations.

2 Intuition and Core Idea

Suppose the true output is y and a machine learning model predicts $\hat{y}$. The residual is

$$r = y - \hat{y}.$$

A loss function determines how much penalty should be assigned to this residual. In squared error, the penalty increases quadratically. Consequently, a few extremely large errors can have a strong influence on the total loss.

Huber Loss introduces a positive threshold called δ. The behaviour of the loss depends on whether the magnitude of the residual is smaller or larger than this threshold.

Key Idea

If $|r| \leq \delta$**:** use a quadratic penalty.
If $|r| > \delta$**:** use a linear penalty.
δ**:** controls the transition between these two regions.

Thus, Huber Loss behaves similarly to squared error for normal prediction errors but prevents unusually large residuals from receiving an excessively large quadratic penalty.

Why is Huber Loss Useful?

Consider two residuals, $r = 1$ and $r = 10$. Under half squared error,

$$L(1) = 0.5, \qquad L(10) = 50.$$

The second observation contributes one hundred times as much loss. If the large residual is caused by an outlier or measurement error, it can significantly influence model training. Huber Loss limits this effect by switching to linear growth after the threshold.

3 Mathematical Formulation

Let

$$r = y - \hat{y}$$

represent the residual between the true value and predicted value. For a threshold $\delta > 0$, Huber Loss is defined as

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta\left(|r| - \frac{1}{2}\delta\right), & |r| > \delta. \end{cases} \tag{1}$$

The first region is quadratic. It applies to relatively small errors. The second region grows linearly with $|r|$ and therefore reduces the effect of large residuals.

At the transition point $|r| = \delta$, the quadratic expression gives

$$\frac{1}{2}\delta^2.$$

The linear expression gives

$$\delta\left(\delta - \frac{1}{2}\delta\right) = \frac{1}{2}\delta^2.$$

Therefore, the two pieces meet continuously.

Derivative of Huber Loss

The derivative with respect to residual r is

$$\frac{dL_\delta}{dr} = \begin{cases} r, & |r| \le \delta, \\ \delta\,\mathrm{sign}(r), & |r| > \delta. \end{cases} \tag{2}$$

For small residuals, the gradient depends directly on r. For large residuals, its magnitude is limited by δ. This is an important reason why a large outlier has less influence than under squared error.

Comparison with Other Regression Losses

Squared error and absolute error are given by

$$L_{\mathrm{SE}}(r) = \frac{1}{2}r^2, \qquad L_{\mathrm{AE}}(r) = |r|.$$

Property	Squared Error	Absolute Error	Huber Loss
Small residuals	Quadratic	Linear	Quadratic
Large residuals	Quadratic	Linear	Linear
Outlier sensitivity	High	Low	Lower than squared error
Differentiable at zero	Yes	No	Yes
Extra parameter	No	No	δ

Table 1: Comparison of three regression loss functions.

Huber Loss therefore occupies a useful middle ground. It retains quadratic behaviour around the optimum while providing linear tails for robustness.

4 Graphical Interpretation

Figure 1 compares half squared error, absolute error, and Huber Loss with $\delta = 1$.

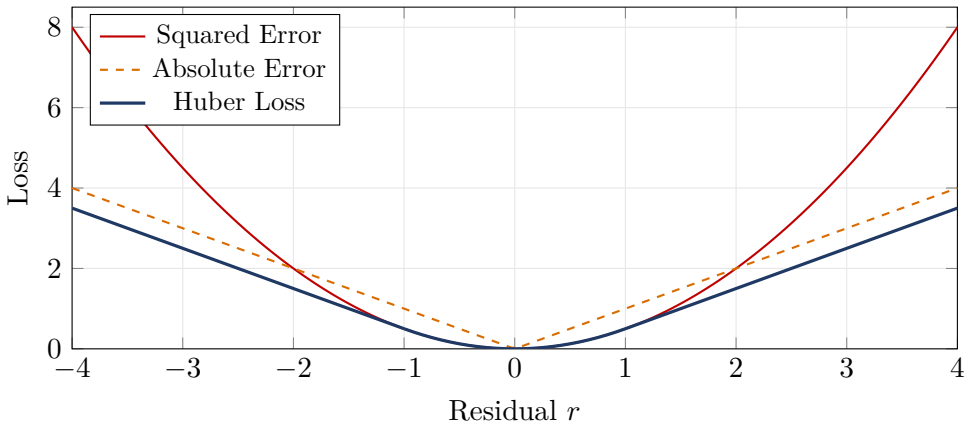


Figure 1: Squared error, absolute error and Huber Loss for $\delta = 1$.

Near $r = 0$, the Huber curve follows squared error. At $r = -1$ and $r = 1$, the function transitions into its linear regions. Beyond these points, the loss continues increasing but does not grow quadratically.

5 Effect of the Delta Parameter

The threshold δ determines where this transition occurs.

A smaller δ causes the loss to enter the linear region earlier, making it more resistant to large residuals. A larger δ extends the quadratic region and makes the behaviour closer to squared error for a wider range of errors.

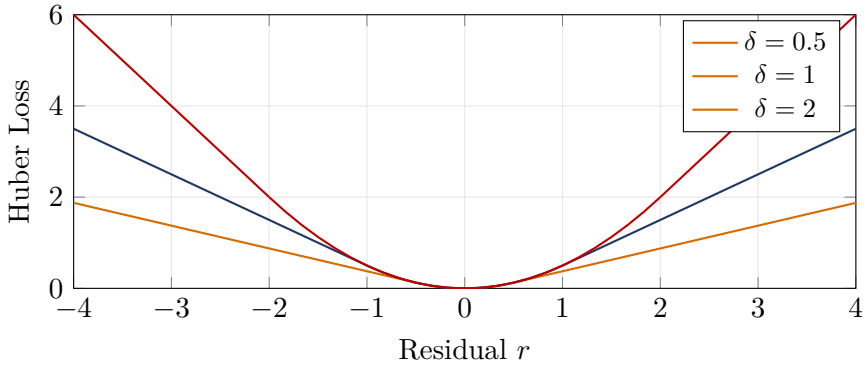


Figure 2: Effect of changing the Huber threshold δ.

The choice of δ should therefore consider the scale of the target variable and the expected magnitude of ordinary prediction errors.

6 Algorithm / Procedure

The Huber Loss calculation requires only the residual and the threshold.

Algorithm 1 Huber Loss Calculation

Require: True value y, predicted value $\hat{y}$, threshold $\delta > 0$
Ensure: Huber Loss L

$r \leftarrow y - \hat{y}$
if $|r| \leq \delta$ **then**
    $L \leftarrow \frac{1}{2}r^2$
else
    $L \leftarrow \delta\left(|r| - \frac{1}{2}\delta\right)$
end if
return L

For n observations, the average loss can be calculated as

$$J = \frac{1}{n}\sum_{i=1}^{n} L_\delta(y_i - \hat{y}_i).$$

During model training, the parameters of the prediction function are adjusted to minimize J.

7 Python Implementation

The following program implements Huber Loss for a single observation.

Listing 1: Python implementation of Huber Loss

```python
def huber_loss(y_true, y_pred, delta=1.0):
    # Calculate residual
    error = y_true - y_pred

    # Magnitude of residual
    abs_error = abs(error)

    # Quadratic region
    if abs_error <= delta:
        loss = 0.5 * error ** 2

    # Linear region
    else:
        loss = delta * (
            abs_error - 0.5 * delta
        )

    return loss


actual = 10
predicted = 12
delta = 1.0

loss = huber_loss(
    actual,
    predicted,
    delta
)

print("Huber Loss =", loss)
```

The computational cost for one observation is constant. For a dataset containing n observations, calculating the loss for every observation requires $O(n)$ operations.

Using NumPy

For multiple predictions, vectorized NumPy operations can be used.

```
import numpy as np

def huber(y_true, y_pred, delta=1.0):
    error = y_true - y_pred
    abs_error = np.abs(error)

    return np.where(
        abs_error <= delta,
        0.5 * error ** 2,
        delta * (abs_error - 0.5 * delta)
    )
```

8 Worked Example

Consider the following values:

$$y = 10, \qquad \hat{y} = 12, \qquad \delta = 1.$$

Step 1: Calculate the residual

$$r = y - \hat{y} = 10 - 12 = -2.$$

Step 2: Calculate its magnitude

$$|r| = |-2| = 2.$$

Step 3: Compare it with δ

$$2 > 1.$$

Therefore, the residual belongs to the linear region.

Step 4: Apply the Huber formula

$$L_\delta(r) = \delta\left(|r| - \frac{1}{2}\delta\right).$$

Substituting the values,

$$L_1(-2) = 1\left(2 - \frac{1}{2}\right) = 1.5.$$

Hence,

$$\boxed{L_1(-2) = 1.5}$$

For comparison, half squared error gives

$$\frac{1}{2}(-2)^2 = 2.$$

Therefore, Huber Loss gives a smaller penalty for this relatively large residual.

Step	Calculation	Result
1	$10 - 12$	$r = -2$
2	$\lvert -2\rvert$	$\lvert r\rvert = 2$
3	$2 > 1$	Linear region
4	$1(2 - 0.5)$	1.5

Table 2: Calculation of Huber Loss for the worked example.

9 Software Demonstration

The following Python program plots Huber Loss together with squared and absolute error.

```
import numpy as np
import matplotlib.pyplot as plt

def huber(error, delta=1.0):
    a = np.abs(error)

    return np.where(
        a <= delta,
```

```
        0.5 * error ** 2,
        delta * (a - 0.5 * delta)
    )

errors = np.linspace(-4, 4, 400)

squared = 0.5 * errors ** 2
absolute = np.abs(errors)
huber_values = huber(errors, 1.0)

plt.plot(errors, squared,
         label="Squared Error")
plt.plot(errors, absolute,
         label="Absolute Error")
plt.plot(errors, huber_values,
         label="Huber Loss")

plt.xlabel("Residual")
plt.ylabel("Loss")
plt.title("Regression Loss Functions")
plt.grid(True)
plt.legend()
plt.show()
```

The value of δ can be changed in the program to observe how the quadratic-to-linear transition moves.

10 Analysis

The principal advantage of Huber Loss is robustness. Squared error can assign extremely high importance to an observation with a large residual. Consider $r = 10$.

Under half squared error,

$$L_{\text{SE}}(10) = \frac{1}{2}(10)^2 = 50.$$

Using Huber Loss with $\delta = 1$,

$$L_1(10) = 1(10 - 0.5) = 9.5.$$

Thus, the large observation still receives a significant penalty, but its effect is much smaller than the quadratic penalty of squared error.

Huber Loss is also differentiable everywhere. This distinguishes it from absolute error, which has a sharp point at zero. Its gradient changes smoothly from the quadratic region to a constant-magnitude linear region.

11 Applications

1. Robust Regression. Huber Loss is useful when regression data contains outliers. It allows a model to fit ordinary observations without allowing a small number of extreme residuals to dominate training.

2. Computer Vision. Huber-type losses can be used for regression quantities such as coordinates and localization values. Robust penalties can be useful when some predictions have unusually large localization errors.

3. Deep Learning. Neural networks performing regression can use Huber Loss as an alternative to MSE when training data contains noisy target values.

4. Sensor and Real-World Data. Sensor datasets can contain abnormal readings caused by temporary faults or measurement noise. Robust losses can reduce the influence of such extreme observations.

5. Forecasting. Time-series data may contain sudden spikes. Huber Loss can be useful when such large deviations should not completely dominate the fitting objective.

12 Advantages and Limitations

Advantages	**Limitations**
More robust to outliers than MSE	Requires selection of δ
Quadratic behaviour for small errors	Choice of δ depends on data scale
Linear growth for large errors	Poor tuning can reduce its advantages
Differentiable everywhere	Adds an extra hyperparameter
Limits large gradient contributions	Not automatically optimal for every dataset

Table 3: Advantages and limitations of Huber Loss.

13 Conclusion

Huber Loss is an important robust loss function used in machine learning, particularly for regression problems. It combines two different behaviours: quadratic penalization for small residuals and linear penalization for large residuals.

The transition is controlled by the threshold parameter δ. When

$$|y - \hat{y}| \leq \delta,$$

Huber Loss behaves like squared error. When

$$|y - \hat{y}| > \delta,$$

the loss grows linearly. This reduces the influence of outliers compared with ordinary squared error.

In the worked example, the actual value was 10, the predicted value was 12, and $\delta = 1$. The residual was -2, giving

$$\boxed{L = 1.5}.$$

The report also demonstrated that δ is an important parameter. Smaller values cause the linear region to begin earlier, while larger values extend the quadratic region.

Overall, Huber Loss provides a practical compromise between squared and absolute error. Its robustness, differentiability, and simple mathematical form make it useful for many machine-learning regression problems involving noisy data or unusual observations.

Declaration of AI Tool Use

Tools used: ChatGPT (OpenAI), Overleaf / LaTeX.

Purpose: Assistance with report structure, explanations, mathematical formatting, LaTeX formatting, graphs, and Python code.

Approximate share of AI-generated content: 20%.

I confirm that I have reviewed the mathematical concepts, code, explanations, and results presented in this report and have disclosed the use of AI tools according to the applicable course requirements.

References

References

[1] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] https://www.geeksforgeeks.org/machine-learning/huber-loss-function-in-machine-learning/

[4] https://en.wikipedia.org/wiki/Huber_loss